

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

PRINKAL SHAHI (1BF24CS229)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **PRINKAL SHAHI (1BF24CS229)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-7
2	Implement Johnson Trotter algorithm to generate permutations.	8-11
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	12-14
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	15-17
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	18-20
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	21-24
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	25-28
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	29-34
9	Implement Fractional Knapsack using Greedy technique.	35-38
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	39-40
11	Implement "N-Queens Problem" using Backtracking.	41-42

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab Program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int adj[MAX][MAX];
int indegree[MAX];
int queue[MAX];
int front = -1, rear = -1;
int V;

void enqueue(int x) {
    if (rear == MAX - 1) return;
    if (front == -1)
        front = 0;
    queue[++rear] = x;
}

int dequeue() {
    if (front == -1 || front > rear) return -1;
    return queue[front++];
}

void topologicalSort() {
    int i, j, count = 0;

    // Initialize indegree of all vertices
    for (i = 0; i < V; i++)
        indegree[i] = 0;

    // Compute indegree
    for (i = 0; i < V; i++) {
        for (j = 0; j < V; j++) {
            if (adj[i][j] == 1)
                indegree[j]++;
        }
    }

    // Enqueue vertices with indegree 0
    for (i = 0; i < V; i++) {
        if (indegree[i] == 0)
            enqueue(i);
    }

    printf("Topological Order: ");
    while (front != -1 && front <= rear) {
```

```

    int u = dequeue();
    printf("%d ", u);
    count++;

    for (j = 0; j < V; j++) {
        if (adj[u][j] == 1) {
            indegree[j]--;
            if (indegree[j] == 0)
                enqueue(j);
        }
    }
}

if (count != V) {
    printf("\nGraph has a cycle! Topological sort not possible.\n");
}

int main() {
    int i, j;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for (i = 0; i < V; i++) {
        for (j = 0; j < V; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    topologicalSort();
    return 0;
}

```

```

Enter number of vertices: 7
Enter adjacency matrix:
0 1 1 0 0 0 0
0 0 0 1 0 1
0 0 0 0 1 0
1 1 1 0 0 1 1
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 1 1 0

Topological Order: 3 0 1 2 6 4 5

```

Leet Code exercises on topological sorting.

207. Course Schedule

```
class Solution { public:
```

```
    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
```

```
        vector<vector<int>>
```

```
            adj(numCourses); vector<int>
```

```
            indegree(numCourses, 0);
```

```
        // Build graph for (auto &p : prerequisites)
```

```
        { int a = p[0];
```

```
          int b = p[1];
```

```
          adj[b].push_back(a);
```

```
          indegree[a]++;
```

```
        }
```

```
        queue<int> q;
```

```
        // Push nodes with indegree 0 for (int
```

```
        i = 0; i < numCourses; i++) { if
```

```
        (indegree[i] == 0) {
```

```
            q.push(i);
```

```
        }
```

```
    }
```

```
    int count = 0;
```

```

while (!q.empty()) { int
    node = q.front();
    q.pop();

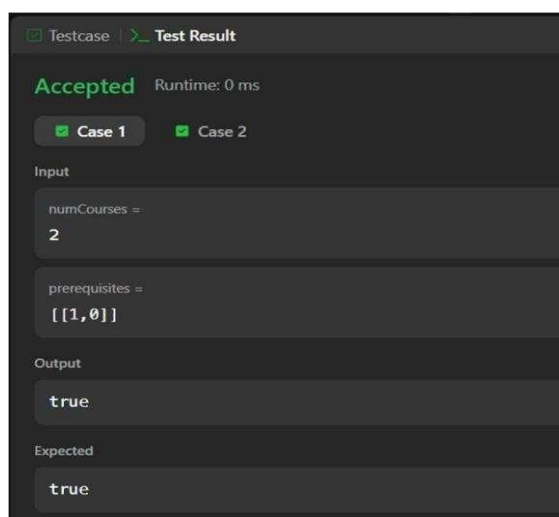
    count++;

    for (int neighbor : adj[node]) {
        indegree[neighbor]--;

        if (indegree[neighbor] == 0) {
            q.push(neighbor);
        }
    }
}

return count == numCourses;
}
};

```



Lab Program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

#include <stdlib.h>

// Directions

#define LEFT -1
#define RIGHT 1

// Structure to store number and its direction
typedef struct { int value; int dir;
} Element;

// Function to print current permutation void
printPermutation(Element arr[], int n) { //
Print arrows (directions) for (int i = 0; i < n;
i++) { if (arr[i].dir == LEFT)
printf("<- "); else printf("-
> ");
}
printf("\n");

// Print values for (int i =
0; i < n; i++) {
printf("%2d ", arr[i].value);
}
printf("\n\n");
}
```



```

// Find the largest mobile element
int getMobile(Element arr[], int n) {
int mobile_index = -1; int
mobile_value = 0;

for (int i = 0; i < n; i++) { int next
    = i + arr[i].dir;

    if (next >= 0 && next < n) { if (arr[i].value > arr[next].value &&
        arr[i].value > mobile_value) { mobile_value = arr[i].value;
        mobile_index = i;
        }
    }
}
return
mobile_index;
}

```

```

// Reverse direction of elements greater than given value
void reverseDirection(Element arr[], int n, int value) {
for (int i = 0; i < n; i++) { if (arr[i].value > value) {
arr[i].dir *= -1;
    }
}
}

```

```

// Johnson-Trotter function void johnsonTrotter(int
n) {
    Element *arr = (Element *)malloc(n * sizeof(Element));

```

```

// Initialize elements for
(int i = 0; i < n; i++) {
    arr[i].value = i + 1;
    arr[i].dir = LEFT;
}

printPermutation(arr, n);

while (1) {
    int mobile_index = getMobile(arr, n);

    if (mobile_index == -1) break;

    int swap_index = mobile_index + arr[mobile_index].dir;

    // Swap
    Element temp = arr[mobile_index];
    arr[mobile_index] = arr[swap_index];
    arr[swap_index] = temp;

    // Reverse directions reverseDirection(arr, n,
    arr[swap_index].value);

    printPermutation(arr, n);
}

free(arr);
}

```

```
int main() { int n;  
    printf("Enter n: ");  
    scanf("%d", &n);  
  
    johnsonTrotter(n);  
  
    return 0;  
}
```

```
Enter n: 3  
1 2 3  
1 3 2  
3 1 2  
3 2 1  
2 3 1  
2 1 3
```

Lab program 3:

For all sorting programs, graph (plotted in Excel) to be attached and shown for large values of 'n' how time increases.

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

// Merge function void merge(int arr[], int left, int
mid, int right) { int n1 = mid - left + 1;
    int n2 = right - mid;

    int *L = (int *)malloc(n1 * sizeof(int)); int
    *R = (int *)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i]; for
    (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left; while (i < n1
    && j < n2) { if (L[i] <= R[j])
    arr[k++] = L[i++]; else arr[k++] =
    R[j++];
    }

    while (i < n1) arr[k++] = L[i++]; while
    (j < n2) arr[k++] = R[j++];

    free(L);

    free(R);
```

```
}
```

```
// Recursive merge sort void mergeSort(int  
arr[], int left, int right) { if (left < right) { int  
mid = left + (right - left) / 2; mergeSort(arr,  
left, mid); mergeSort(arr, mid + 1, right);  
merge(arr, left, mid, right);  
}  
}
```

```
int main() { int n; printf("Enter number of items (e.g. 100000 to  
1000000): "); scanf("%d", &n);
```

```
int *arr = (int *)malloc(n * sizeof(int));  
if (arr == NULL) { printf("Memory  
allocation failed!\n"); return 1;  
}
```

```
srand(time(NULL));  
for (int i = 0; i < n; i++) { arr[i]  
= rand();  
}
```

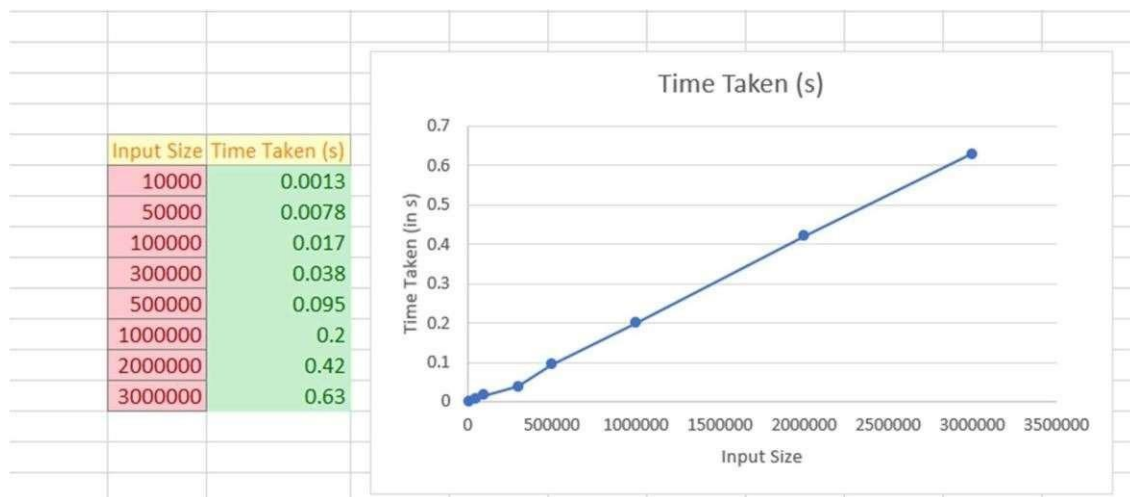
```
clock_t start = clock();  
mergeSort(arr, 0, n - 1); clock_t  
end = clock();
```

```
double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC; printf("\nTime taken  
to sort %d elements: %.9f seconds\n", n, time_taken);
```

```
free(arr); return
```

```
0;
```

```
}
```



Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
// Swap function void
```

```
swap(int *a, int *b) { int temp
```

```
= *a; *a = *b;
```

```
    *b = temp;
```

```
}
```

```
// Partition function int partition(int
```

```
arr[], int low, int high) { int pivot =
```

```
arr[high];
```

```
    int i = (low - 1);
```

```
    for (int j = low; j < high; j++) {
```

```
        if (arr[j] <= pivot) { i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
    } swap(&arr[i + 1],
```

```
    &arr[high]); return (i + 1);
```

```
}
```

```
// QuickSort function
```

```
void quickSort(int arr[], int low, int high) { if
```

```
    (low < high) {
```

```

        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1); quickSort(arr, pi
        + 1, high);
    }
}

```

```

int main() { int n; printf("Enter number of
elements: "); scanf("%d", &n);

```

```

    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) { printf("Memory
allocation failed!\n"); return 1;
}

```

```

    srand(time(NULL));
    for (int i = 0; i < n; i++) { arr[i] = rand(); // fill
        with random numbers
    }

```

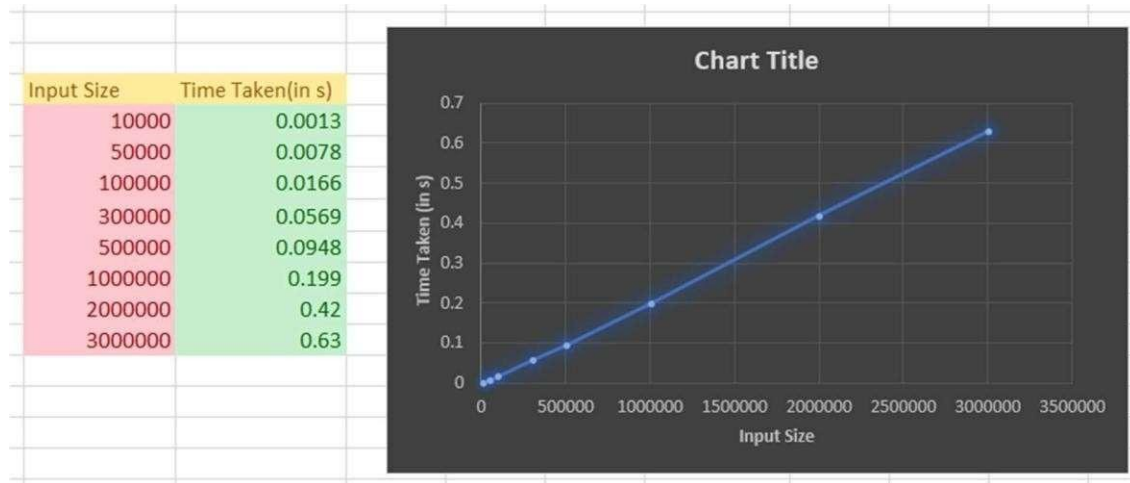
```

    clock_t start = clock();
    quickSort(arr, 0, n - 1);
    clock_t end = clock();
    double time_taken =
        ((double)(end - start)) /
        CLOCKS_PER_SEC;
    printf("QuickSort    on %d
elements took %.6f
seconds\n",        n,
        time_taken);

```



```
free(arr); return  
0;  
}
```



Lab Program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
```

```
void bottomUpHeapify(int arr[], int n, int i)
```

```
{ int temp = arr[i]; int
```

```
    child = 2 * i + 1;
```

```
    while (child < n)
```

```
    { if (child + 1 < n && arr[child + 1] > arr[child])
```

```
        child++;
```

```
        if (arr[child] > temp)
```

```
        { arr[i] = arr[child];
```

```
          i = child; child =
```

```
            2 * i + 1;
```

```
        } else
```

```
        { break;
```

```
        } } arr[i] =
```

```
temp;
```

```
}
```

```
void buildMaxHeap(int arr[], int n)
```

```
{ for (int i = n / 2 - 1; i >= 0; i--)
```

```
    { bottomUpHeapify(arr, n, i);
```

```
    }
```

```
}
```

```
void heapSort(int arr[], int n)
```

```

{
    buildMaxHeap(arr, n);

    for (int i = n - 1; i > 0; i--)
    { int temp = arr[0];
      arr[0] = arr[i];
      arr[i] = temp;

      bottomUpHeapify(arr, i, 0);
    }
}

void printArray(int arr[], int n)
{ for (int i = 0; i < n; i++)
    printf("%d ", arr[i]); printf("\n");
}

int main()
{ int n; printf("Enter number of
  elements: "); scanf("%d", &n);

  int arr[n]; printf("Enter %d
  elements:\n", n); for (int i = 0; i < n;
  i++)
  { scanf("%d", &arr[i]);
  }

  printf("\nOriginal array:\n");
  printArray(arr, n); heapSort(arr, n);

```

```
printf("\nSorted array:\n"); printArray(arr,  
n); return 0;  
}
```

```
Enter number of elements: 6  
Enter 6 elements:  
63  
99  
54  
3  
18  
21
```

```
Original array:  
63 99 54 3 18 21
```

```
Sorted array:  
3 18 21 54 63 99
```

Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>

int max(int a, int b)
{ return (a > b) ? a : b;
}

int knapsack(int W, int wt[], int val[], int n)
{ int i, w;
  int K[n + 1][W + 1];

  for (i = 0; i <= n; i++)
  {
    for (w = 0; w <= W; w++)
    { if (i == 0 || w == 0)
      {
        K[i][w] = 0; } else
      if (wt[i - 1] <=
        w)
      {
        K[i][w] = max( val[i - 1] + K[i -
          1][w - wt[i - 1]],
          K[i - 1][w]
        );
      } else
      {
```

```

        K[i][w] = K[i - 1][w];
    }
}
}      return
K[n][W];
}

int main()
{
    int n, W;

    // 1. Get the number of items and capacity
    printf("Enter the number of items: "); scanf("%d",
    &n);

    printf("Enter the maximum capacity of the knapsack: ");
    scanf("%d", &W);

    int val[n], wt[n];

    // 2. Get weights and values from user
    printf("\n--- Enter Item Details ---\n");
    for (int i = 0; i < n; i++)
    { printf("Item %d - Value: ", i + 1); scanf("%d",
        &val[i]); printf("Item %d - Weight: ", i + 1);
        scanf("%d",
        &wt[i]);
    }

    // 3. Calculate and display result

```

```
int result = knapsack(W, wt, val, n);

printf("\n\n");

printf("Maximum value in Knapsack = %d\n", result);

printf("\n\n");

return 0;
}
```

```
Enter the number of items: 3
Enter the maximum capacity of the knapsack: 50

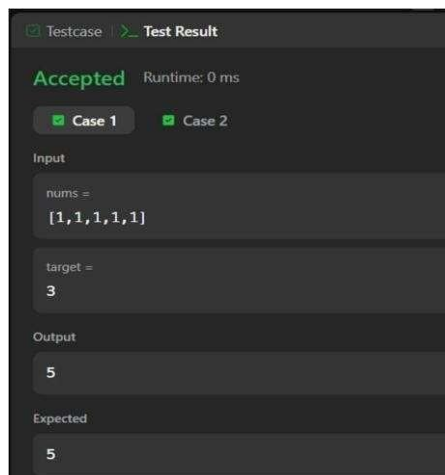
--- Enter Item Details ---
Item 1 - Value: 60
Item 1 - Weight: 10
Item 2 - Value: 100
Item 2 - Weight: 20
Item 3 - Value: 120
Item 3 - Weight: 30

-----
Maximum value in Knapsack = 220
-----
```

Leet Code exercise on Knapsack problem using dynamic programming.

494. Target Sum

```
class Solution { public:
    int findTargetSumWays(vector<int>& nums, int
        target){ int total = 0; for (int num : nums) total +=
        num; if (abs(target) > total || (target + total) % 2 != 0)
        return 0; int sum = (target + total) / 2; vector<int>
        dp(sum + 1, 0); dp[0] = 1; for (int num : nums) { for
        (int j = sum; j >= num; j--) { dp[j] += dp[j - num];
            }
        }
        return
        dp[sum];
    }
};
```



Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>

#define INF 99999

int main() {
    int n, i, j, k;

    printf("Enter number of vertices: "); scanf("%d",
    &n); int dist[n][n]; printf("Enter the adjacency
    matrix:\n"); printf("(Enter %d if there is no direct
    edge)\n", INF); for(i = 0; i < n; i++) { for(j = 0; j
    < n; j++) { printf("Distance from %d to %d: ", i,
    j); scanf("%d", &dist[i][j]);
        }
    } for(k = 0; k < n; k++) { for(i = 0; i < n; i++)
    { for(j = 0; j < n; j++) { if(dist[i][k] +
    dist[k][j] < dist[i][j]) { dist[i][j] = dist[i][k] +
    dist[k][j];
        }
    }
    }

    } printf("\nShortest distance
    matrix:\n"); for(i = 0; i < n; i++) { for(j
    = 0; j < n; j++) { if(dist[i][j] == INF)
    printf("INF "); else printf("%d ",
```

```
dist[i][j]);  
    }  
    printf("\n"); }  
    return 0;  
}
```

```
Enter number of vertices: 4  
Enter the adjacency matrix:  
(Enter 99999 if there is no direct edge)  
Distance from 0 to 0: 0  
Distance from 0 to 1: 4  
Distance from 0 to 2: 2  
Distance from 0 to 3: 4  
Distance from 1 to 0: 4  
Distance from 1 to 1: 1  
Distance from 1 to 2: 3  
Distance from 1 to 3: 1  
Distance from 2 to 0: 99999  
Distance from 2 to 1: 3  
Distance from 2 to 2: 99999  
Distance from 2 to 3: 2  
Distance from 3 to 0: 4  
Distance from 3 to 1: 1  
Distance from 3 to 2: 2  
Distance from 3 to 3: 99999  
  
Shortest distance matrix:  
0 4 2 4  
4 1 3 1  
6 3 4 2  
4 1 2 2
```

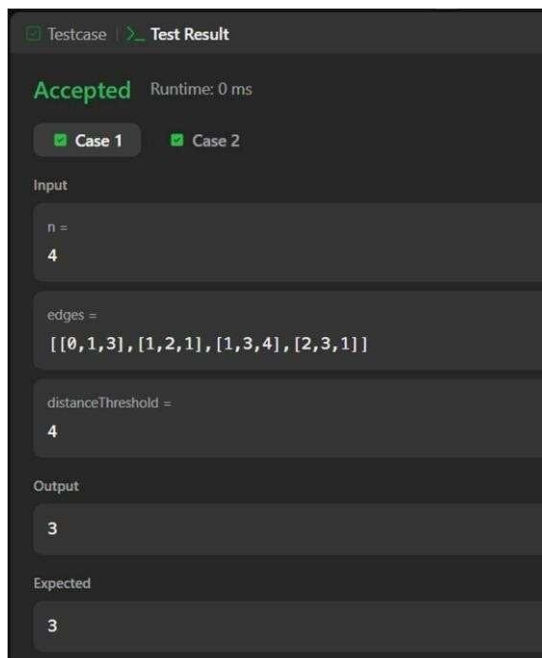
Leet Code exercise on Floyd's algorithm.

1334. Find the city with the smallest number of neighbors at a threshold distance.

class Solution { public:

```
    int findTheCity(int n, vector<vector<int>>& edges, int distanceThreshold) {  
        vector<vector<int>> dist(n, vector<int>(n, 1e9));  
        // Distance to self = 0 for (int  
        i = 0; i < n; i++) {  
            dist[i][i] = 0;  
            } for (auto &e : edges) {  
                int u = e[0]; int v = e[1];  
                int w = e[2]; dist[u][v]  
                = w;  
                dist[v][u] = w;  
            } for (int via = 0; via < n; via++) { for  
            (int i = 0; i < n; i++) { for (int j  
            = 0; j < n; j++) { if (dist[i][via] != 1e9  
            && dist[via][j] != 1e9) { dist[i][j] =  
            min(dist[i][j],  
                dist[i][via] + dist[via][j]);  
            }  
            }  
            }  
        }  
  
        int minCount = n; int ans = -1; for (int i =  
        0; i < n; i++) { int count = 0; for (int j =  
        0; j < n; j++) { if (dist[i][j] <=  
        distanceThreshold) { count++;  
        }  
        }
```

```
        } if (count <= minCount)
        { minCount = count;
          ans = i;
        } } return
ans;
}
};
```



Lab Program 8a:

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>
```

```
#define MAX 100
```

```
int parent[MAX];
```

```
int find(int i) {  
    while(parent[i] != i) i  
    = parent[i]; return  
    i;  
}
```

```
void unionSet(int i, int j) { int  
    a = find(i); int b = find(j);  
    parent[a] = b;  
}
```

```
int main() { int  
    V, i, j;
```

```
    printf("Enter number of vertices: "); scanf("%d",  
    &V);
```

```
    int cost[MAX][MAX];
```

```
    printf("Enter cost adjacency matrix:\n"); for(i  
    = 0; i < V; i++) {
```

```

        for(j = 0; j < V; j++) { scanf("%d",
            &cost[i][j]);
            if(cost[i][j] == 0)
                cost[i][j] = 9999;
        }
    }
}

```

```

for(i = 0; i < V; i++) parent[i]
    = i;

```

```

int edges = 0, mincost = 0;

```

```

printf("Edges in MST:\n");

```

```

while(edges < V - 1) {
    int min = 9999, a = -1, b = -1;

```

```

    // Find minimum edge

```

```

    for(i = 0; i < V; i++) {
        for(j = 0; j < V; j++) {
            if(cost[i][j] < min) { min
                = cost[i][j]; a = i; b = j;
            }
        }
    }
}

```

```

int u = find(a);

```

```

int v = find(b);

```

```

        if(u != v) { printf("%d - %d : %d\n", a,
            b, min); unionSet(u, v); mincost +=
            min; edges++;
        }

        cost[a][b] = cost[b][a] = 9999;
    }

    printf("Total cost = %d\n", mincost); return
    0;
}

```

```

Enter number of vertices: 4
Enter cost adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0
Edges in MST:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Total cost = 19

```

Lab Program 8b:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define V 5

int minKey(int key[], bool mstSet[]) { int min
    = INT_MAX, min_index;

    for (int v = 0; v < V; v++) { if
        (!mstSet[v] && key[v] < min) {
            min = key[v]; min_index = v;
        }
    }
    return
    min_index;
}

void printMST(int parent[], int graph[V][V]) { printf("Edge
    \tWeight\n"); for (int i = 1; i < V; i++) { printf("%d - %d
    \t%d\n", parent[i], i, graph[i][parent[i]]);
    }
}

void primMST(int graph[V][V]) {
    int parent[V], total = 0; int key[V];
```



```

bool mstSet[V];

for (int i = 0; i < V; i++) {
    key[i] = INT_MAX;
    mstSet[i] = false;
}

key[0] = 0; parent[0] = -1; for (int count = 0;
count < V - 1; count++) { int u =
minKey(key, mstSet);
    mstSet[u] = true;

    for (int v = 0; v < V; v++) { if (graph[u][v] && !mstSet[v] &&
        graph[u][v] < key[v]) { parent[v] = u; key[v] = graph[u][v];
        }
    }
}

printf("Edge \tWeight\n"); for (int i = 1; i < V; i++) {
    printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
    total += graph[i][parent[i]];
}

printf("Total weight of MST = %d\n", total);

}

int main() { int

```

```
graph[V][V] = {  
    {0, 2, 0, 6, 0},  
    {2, 0, 3, 8, 5},  
    {0, 3, 0, 0, 7},  
    {6, 8, 0, 0, 9},  
    {0, 5, 7, 9, 0}  
};  
  
primMST(graph);  
  
return 0;  
}
```

Edge	Weight
0 - 1	2
1 - 2	3
0 - 3	6
1 - 4	5
Total weight of MST = 16	

Lab Program 9:

Implement fractional knapsack problem using Greedy technique.

```
#include <stdio.h>

struct Item {
    int id; int
    profit; int
    weight;
    float ratio;
};

// Function to sort items based on ratio void sortItems(struct
Item items[], int n) {
    int i, j;
    struct Item temp;

    for(i = 0; i < n - 1; i++) { for(j = i + 1;
        j < n; j++) { if(items[i].ratio <
        items[j].ratio) { temp = items[i];
        items[i] = items[j]; items[j] = temp;
        }
        }
    }
}

// Fractional Knapsack Function
float fractionalKnapsack(struct Item items[], int n, int capacity) {
```

```

int
i;

float totalProfit = 0.0;

// Sort items by ratio sortItems(items,
n);

printf("\nSelected                Items:\n");
printf("Item\tProfit\tWeight\tFraction Taken\n");

for(i = 0; i < n; i++) {

    // Take complete item if(items[i].weight
    <= capacity) {

        capacity -= items[i].weight; totalProfit
        += items[i].profit;

        printf("%d\t%d\t%d\t1.00\n",
            items[i].id, items[i].profit,
            items[i].weight);
    }

    // Take fractional item else
    {

        float fraction = (float)capacity / items[i].weight;

        totalProfit += items[i].profit * fraction;
    }
}

```

```

        printf("%d\t%d\t%d\t%.2f\n",
               items[i].id, items[i].profit,
               items[i].weight,
               fraction);

        break;
    }
}

return totalProfit;
}

int main() {

    int n, capacity, i;

    printf("Enter number of items: "); scanf("%d",
    &n);

    struct Item items[n];

    // Input profits and weights for(i
    = 0; i < n; i++) {

        items[i].id = i + 1;

```

```

printf("\nEnter profit for item %d: ", i + 1);
scanf("%d", &items[i].profit);
printf("Enter weight for item %d: ", i + 1);
scanf("%d", &items[i].weight);
items[i].ratio =
    (float)items[i].profit / items[i].weight; }
printf("\nEnter knapsack capacity: ");
scanf("%d", &capacity); float maxProfit =
fractionalKnapsack(items, n, capacity);
printf("\nMaximum Profit = %.2f\n",
maxProfit); return 0;
}

```

```

Enter number of items: 3

Enter profit for item 1: 60
Enter weight for item 1: 10

Enter profit for item 2: 100
Enter weight for item 2: 20

Enter profit for item 3: 120
Enter weight for item 3: 30

Enter knapsack capacity: 50

Selected Items:

```

Item	Profit	Weight	Fraction Taken
1	60	10	1.00
2	100	20	1.00
3	120	30	0.67

```

Maximum Profit = 240.00

```

Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>

#define V 5
#define INF 99999

// Function to find the vertex with minimum distance
int minDistance(int dist[], int visited[]) {
    int min = INF, min_index = -1;

    for (int v = 0; v < V; v++) {
        if (visited[v] == 0 && dist[v] < min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

// Function to print shortest distances
void printSolution(int dist[]) {
    printf("Vertex\tDistance from Source\n");
    for (int i = 0; i < V; i++) {
        printf("%d\t%d\n", i, dist[i]);
    }
}

// Dijkstra's Algorithm
void dijkstra(int graph[V][V], int src) {
    int dist[V];
    int visited[V];

    // Initialize distances and visited array
    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }
    dist[src] = 0;

    // Find shortest path for all vertices
    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited);
```

```

    visited[u] = 1;

    // Update distance value of adjacent vertices
    for (int v = 0; v < V; v++) {
        if (!visited[v] && graph[u][v] && dist[u] != INF &&
            dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }
}

printSolution(dist);
}

int main() {
    int graph[V][V] = {
        {0, 10, 0, 30, 100},
        {10, 0, 50, 0, 0},
        {0, 50, 0, 20, 10},
        {30, 0, 20, 0, 60},
        {100, 0, 10, 60, 0}
    };

    int source = 0;
    dijkstra(graph, source);

    return 0;
}

```

Vertex	Distance from Source
0	0
1	10
2	50
3	30
4	60

Program 11:
Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>

#define MAX 10

int board[MAX][MAX];
int n;

void printBoard() {
    int i, j;

    printf("\nSolution:\n");

    for(i = 0; i < n; i++) { for(j =
        0; j < n; j++) {
        if(board[i][j] == 1)
            printf("Q ");    else
            printf(". ");    }
        printf("\n");
    }
}

int isSafe(int row, int col) {
    int i, j;

    for(i = 0; i < row; i++) { if(board[i][col] == 1) return
0; } for(i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)
```

```

{ if(board[i][j] == 1) return 0; } for(i = row - 1, j = col +
1; i >= 0 && j < n; i--, j++) { if(board[i][j] == 1) return
0; } return 1; } int solveNQueens(int row) {
    int col; if(row == n) { printBoard();
        return 1; } for(col = 0;
col < n; col++) {
    if(isSafe(row, col)) {
        board[row][col] = 1;
        solveNQueens(row + 1);
        board[row][col] = 0;
    } }
    return 0;
}
int main() { printf("Enter number of queens:
"); scanf("%d", &n);
    solveNQueens(0); return 0;
}

```

```

Enter number of queens: 6

Solution:
. Q . . . .
. . . Q . .
. . . . . Q
Q . . . . .
. . Q . . .
. . . . Q .

Solution:
. . Q . . .
. . . . . Q
. Q . . . .
. . . . Q .
Q . . . . .
. . . Q . .

Solution:
. . . Q . .
Q . . . . .
. . . . Q .
. Q . . . .
. . Q . . .
. . . . . Q

Solution:
. . . . Q .
. . Q . . .
Q . . . . .
. . . . . Q
. . . Q . .
. Q . . . .

```